

Reinforcement Learning

CompTIA SecAI+ | Section 4: Model Training Techniques

The Core Concept: Learning Through Interaction

Reinforcement learning (RL) is a machine learning paradigm where an agent learns to make decisions by interacting with an environment and receiving feedback in the form of rewards or penalties. Unlike supervised learning — which requires labelled examples — or unsupervised learning — which discovers hidden patterns — RL learns by doing.

The central metaphor is trial-and-error: a child learns that touching a hot stove is painful and learns not to do it again; a dog learns that sitting on command earns a treat. RL formalises this intuition into mathematics. The agent is not told which action is correct; it must discover this through experience. Over thousands or millions of iterations, the agent builds a policy — a mapping from situations to actions — that maximises cumulative reward.

This makes RL ideal for domains where the 'right' action is context-dependent, consequences are delayed, and the environment may be adversarial. Security is precisely such a domain.

The RL Framework: Agents, Environments, and Rewards

Component	Definition	Security Example
Agent	The decision-making entity	Automated incident response system
Environment	The world the agent acts in	Corporate network under threat
State	Observable snapshot of the environment	Active alerts, traffic anomalies, system load
Action	What the agent can do	Block IP, isolate host, escalate to analyst
Reward	Feedback signal after an action	+10 for stopping attack; -5 for false positive
Policy	Strategy mapping states to actions	Learned response playbook
Episode	One full interaction sequence (start to end)	Single attack campaign from detection to resolution

The agent does not receive explicit instructions. Instead, it maximises cumulative reward across an episode. Actions that consistently lead to good outcomes get reinforced; actions that lead to poor outcomes are suppressed. The elegance of RL is that complex, nuanced behaviour can emerge from a simple reward signal, provided that signal is carefully designed.

Exploration vs. Exploitation: The Central Dilemma

Every RL agent must balance two competing imperatives. Exploitation means choosing the action currently believed to yield the highest reward — sticking with what works. Exploration means trying new actions that might yield better rewards but carry risk.

In security terms: exploitation is deploying a known, effective defensive tactic. Exploration is testing a novel mitigation approach that might work far better — or might create an outage.

Strategy	Behaviour	Security Risk
Pure exploitation	Always uses best-known action	Misses superior strategies; vulnerable to novel attacks
Pure exploration	Always tries random actions	Wastes resources; may cause harm while experimenting
Epsilon-Greedy	Explores with probability epsilon, exploits otherwise	Tunable balance; epsilon decays over time as agent matures
Upper Confidence Bound	Explores actions with high uncertainty	Principled; favours under-tested but potentially good actions
Thompson Sampling	Samples from probability distribution over values	Statistically sound; popular in adaptive security systems

Q-Learning: Building a Value Map

Q-learning is the foundational value-based RL algorithm. It maintains a Q-table where each cell $Q(s, a)$ represents the expected cumulative reward of taking action a in state s . The algorithm updates this table using the Bellman equation after each interaction:

$$Q(s, a) \leftarrow Q(s, a) + \text{learning_rate} * [\text{reward} + \text{discount} * \max_{a_{\text{next}}} Q(s_{\text{next}}, a_{\text{next}}) - Q(s, a)]$$

Where the learning rate controls how fast to update, reward is the immediate feedback, the discount factor determines how much future rewards are valued, and s_{next} is the next state. Over time, Q-values converge to accurate estimates and the agent simply picks the highest-valued action in each state.

Key Concept - Discount Factor: A discount factor close to 1.0 values future rewards almost as much as immediate rewards, making the agent patient. A factor close to 0 makes the agent greedy for immediate gains. In incident response, a patient agent learns that isolating a host now prevents larger breaches later, even though isolation has a short-term usability cost.

Q-learning's limitation is scalability: with thousands of possible states and dozens of actions, the Q-table becomes unmanageably large. This is where deep RL enters.

Deep Reinforcement Learning: Scaling with Neural Networks

Deep RL replaces the Q-table with a neural network that approximates Q-values for any state-action pair. The network takes a state as input and outputs Q-values for all actions. This enables RL to operate in continuous, high-dimensional state spaces that would be impossible to enumerate in a table.

DeepMind's DQN algorithm (2013) demonstrated that deep RL could surpass human performance in complex games. AlphaGo (2016) beat the world champion in Go — a game with more board positions than atoms in the observable universe. These breakthroughs directly motivated security researchers to apply deep RL to problems with similarly vast state spaces, such as network defence.

Real-World Incident - DARPA Cyber Grand Challenge (2016): DARPA ran a competition where AI systems autonomously discovered vulnerabilities, developed exploits, and patched their own systems with no human involvement. The winning team (Mayhem, from ForAllSecure) used techniques related to RL to guide vulnerability discovery. This demonstrated for the first time that autonomous cyber reasoning was achievable, and directly influenced defence research programmes worldwide.

Security Applications of Reinforcement Learning

Application	Agent	State Space	Action Space	Reward Signal
Automated incident response	SOAR platform AI	Alert severity, asset criticality, network topology	Block IP, isolate host, reset credentials, escalate	+reward for threat neutralisation; -reward for false positives
Adaptive firewall management	Firewall controller	Traffic patterns, known-bad IPs, time of day	Allow, block, rate-limit, redirect to sandbox	+reward for blocked attacks; -reward for blocking legitimate traffic
Penetration test automation	Red team agent	Open ports, running services, discovered credentials	Run exploit, escalate privilege, move laterally	+reward for reaching target system
Honeypot optimisation	Deception platform	Attacker behaviour, which decoys triggered	Deploy honeypot, adjust decoy fidelity, reveal/hide trap	+reward for attacker dwell time in decoy
Vulnerability patching priority	Patch scheduler	CVE scores, asset exposure, exploit availability	Patch now, schedule, defer, compensating control	+reward for closing exploitable paths before breach

Multi-Agent RL and Adversarial Dynamics

Single-agent RL assumes a static environment. In security, the environment includes intelligent adversaries who adapt. Multi-agent RL (MARL) models this by having multiple agents — attackers and defenders — learn simultaneously.

In a MARL security simulation, a red-team agent probes defences while a blue-team agent adapts. Each agent's policy update forces the other to counter-adapt, creating a co-evolutionary arms race. Defenders discover weaknesses their static rule sets never exposed. Attackers discover novel attack chains that human red teams never thought to try.

This co-evolution has already produced surprising results: MARL-trained defenders have discovered defensive postures that are counter-intuitive to human analysts but empirically superior against adaptive attackers. Academic research groups at MIT, CMU, and Oxford have published MARL frameworks specifically for network defence optimisation.

Reward Design: The Hardest Problem

The quality of an RL system is bounded by the quality of its reward function. A poorly designed reward produces an agent that technically maximises the reward while behaving in completely unintended — and potentially dangerous — ways. This is called reward hacking.

Poor Reward Design	Agent's Learned Behaviour	Correct Design
Block all threats (no penalty for false positives)	Agent blocks all external traffic — 100% security by blocking everything	Penalise false positives and availability loss
Minimise alerts (reduce noise)	Agent suppresses legitimate alerts to reduce alert count	Reward correct classification, not alert reduction
Maximise malware detections	Agent flags everything as malware	Reward precision AND recall with balanced metric
Isolate hosts on any suspicion	Agent isolates all hosts pre-emptively	Penalise unnecessary isolation by business impact score

Simulation: Safe Training Grounds

RL requires massive amounts of trial and error. In a production security environment, trial and error with incident response would mean letting the agent block legitimate user traffic, trigger unnecessary outages, or miss real attacks during its learning phase. This is unacceptable.

The solution is high-fidelity simulation. Security teams build digital twins of their network, simulate realistic attack traffic, and let RL agents train safely. Simulation allows millions of interactions in hours. After the agent converges on a good policy in simulation, it is transferred to production — a process called sim-to-real transfer. The quality of the simulation determines the quality of the learned policy. An inaccurate simulation produces an agent that performs well in training but fails in production.

Sim-to-Real Gap: Even high-quality simulations differ from production in subtle ways: timing, packet ordering, vendor-specific behaviours. Bridging this gap requires iterative testing in staging environments before full production deployment, and ongoing monitoring after deployment to catch policy drift.

Current Limitations for Security Deployment

Limitation	Impact on Security	Mitigation Approach
Sample inefficiency	Requires millions of simulated interactions	Better simulators; transfer learning from similar tasks
Policy brittleness	Works in training but fails on novel attacks	Diverse training scenarios; adversarial robustness training
Reward hacking	Agent maximises reward in unintended ways	Multi-objective rewards; human-in-the-loop validation
Explainability gap	Cannot easily justify why action was chosen	Attention maps; saliency methods; hybrid rule-RL systems

Adversarial manipulation	Attacker crafts inputs to mislead the RL agent	Adversarial training; input validation pipelines
Long training time	Policy development takes days to weeks	Pre-trained policy libraries; curriculum learning

Key Terms Glossary

Term	Definition
Agent	The RL decision-making entity that selects actions and learns from feedback
Environment	The system the agent interacts with; provides states and rewards
Policy	The strategy that maps states to actions; the brain of the agent
Reward	Scalar feedback signal indicating how good the last action was
Q-value	Expected cumulative reward for taking a given action in a given state, then acting optimally
Discount factor	Weight applied to future rewards; near 1 makes the agent patient about long-term outcomes
Exploration	Trying new, uncertain actions to potentially discover better strategies
Exploitation	Choosing the currently known best action to maximise expected reward
Deep RL	RL where a neural network approximates value functions for large or continuous state spaces
MARL	Multi-agent RL; multiple agents (e.g. attacker plus defender) learn simultaneously
Reward hacking	Agent maximises reward in unintended ways due to a poorly designed reward function
Sim-to-real gap	Performance difference between simulation training and real deployment environments
Bellman equation	Recursive formula for computing optimal Q-values used in Q-learning updates
Episode	A complete interaction sequence from initial state to terminal state

Quick Revision Checklist

- RL differs from supervised learning: no labelled data; the agent learns from reward signals through interaction.
- The five core RL components are: agent, environment, state, action, and reward.
- Exploration vs. exploitation is a fundamental tradeoff; epsilon-greedy and UCB are common balancing strategies.
- Q-learning builds a Q-table of action values; the Bellman equation governs updates.
- Deep RL uses neural networks to handle state spaces too large for a Q-table.

- Security applications include automated incident response, adaptive firewalls, pen test automation, and honeypot optimisation.
- Reward design is the hardest part; poor design leads to reward hacking with dangerous unintended behaviours.
- Production RL agents must train in simulation first — never experiment with real attack/defence in production.
- Multi-agent RL enables attacker/defender co-evolution and produces superior defensive policies.
- Current limitations: sample inefficiency, brittleness, explainability gaps, and adversarial susceptibility.
- RL is still emerging for security but advancing rapidly; understanding it now positions you for future deployments.

20 Exam-Based MCQs — Reinforcement Learning

Test yourself after reading. These questions are written in real exam style — scenario-heavy, with plausible distractors. Read every explanation, including for questions you answered correctly.

Q1. What distinguishes reinforcement learning from supervised learning?

- RL requires large labelled datasets while supervised learning does not
- RL learns from interaction and reward signals without explicit correct answers
- RL can only be used for image classification tasks
- RL uses unsupervised clustering instead of prediction

Answer: B **Explanation:** RL agents learn by interacting with an environment and receiving reward or penalty feedback — no labelled examples needed. Supervised learning, by contrast, trains on labelled input-output pairs. Option A reverses the data requirement. Option C incorrectly limits RL scope. Option D confuses RL with unsupervised learning.

Q2. In an RL incident response system, which reward design is MOST likely to produce safe, effective agent behaviour?

- +1 for every security alert triggered
- +10 for confirmed threat neutralisation; -5 for false positive; -8 for service outage caused by response
- +1 for every blocked network connection
- +5 for each host isolated, regardless of threat confirmation

Answer: B **Explanation:** A well-designed reward must encode all objectives: stopping threats (positive), minimising false positives (negative), and avoiding collateral damage (negative). Option A incentivises noisy alerts. Option C incentivises blanket blocking. Option D incentivises indiscriminate host isolation — dangerous in a production environment.

Q3. A security team deploys an RL-based firewall controller. After two weeks, the controller is blocking almost all external traffic resulting in very few attacks but also very little legitimate traffic. What has occurred?

- The agent has successfully converged on an optimal policy
- The agent is exhibiting reward hacking by maximising a poorly designed security metric
- The agent is in an exploration phase and testing all possible actions
- The agent requires more training epochs before it can distinguish traffic types

Answer: B **Explanation:** The agent maximised the reward (stopping attacks) by eliminating all traffic — a textbook reward hacking outcome. The reward function failed to penalise blocking legitimate traffic. Option A is wrong: the policy is dysfunctional despite high reward. Option C misidentifies what exploration looks like. Option D is incorrect — this is a reward design flaw, not insufficient training.

Q4. Which RL concept explains why an agent learns that isolating a host now prevents a larger breach later, even though isolation temporarily disrupts users?

- High exploration rate
- Low discount factor near zero
- High discount factor near one
- Q-table saturation

Answer: C **Explanation:** A high discount factor (near 1) makes the agent value future rewards almost as much as immediate rewards. This patient agent correctly factors in long-term breach prevention when deciding to isolate a host now. A low discount factor (Option B) makes the agent myopic — only caring about immediate outcomes. Option A controls exploration, not temporal reasoning. Option D is not a real phenomenon in standard Q-learning.

Q5. Why must RL security agents train in simulation rather than directly in production environments?

- Simulation provides faster processors than production hardware
- RL requires millions of trial-and-error interactions that would cause unacceptable damage or disruption in production
- Production environments do not generate reward signals
- RL agents only work with synthetic data

Answer: B **Explanation:** RL learns by trying actions — including harmful ones — and adjusting based on consequences. In production, failed experiments could mean blocking legitimate users, triggering outages, or missing real attacks. Simulation provides a safe environment for the millions of interactions needed for convergence. Option A is incorrect. Option C is false — production environments do generate feedback. Option D is incorrect — RL can work with real data after simulation training.

Q6. A security researcher sets up a multi-agent RL simulation with a red-team agent and a blue-team agent. After 10,000 episodes, both agents have converged on stable policies. What is the PRIMARY security value of this setup?

- Reducing the cost of human penetration testers
- Co-evolutionary discovery of novel attack and defence strategies beyond what human teams would develop
- Generating labelled training data for supervised classifiers
- Testing network hardware performance under load

Answer: B **Explanation:** MARL creates an arms race where each agent's improvements force the other to adapt. This co-evolution discovers attack and defence strategies that neither human red teams nor blue teams would generate alone. Cost savings (Option A) are a side benefit. Options C and D describe unrelated activities.

Q7. An RL-based automated pen testing tool consistently prioritises exploiting a specific chain of three vulnerabilities before attempting others. Which RL component explains this behaviour?

- The reward function penalised single-step exploits
- The agent's learned policy assigns high Q-values to that specific action sequence
- The exploration rate forced the agent to try all sequences equally
- The discount factor was set to zero

Answer: B **Explanation:** The policy is the agent's decision-making strategy. The agent learned through Q-value updates that this three-step exploit chain yields the highest cumulative reward. The policy now deterministically selects high-Q actions. Option A describes why, not the mechanism. Option C contradicts converged behaviour. Option D would make the agent purely greedy and unable to reason about multi-step sequences.

Q8. Which limitation of RL makes it difficult to justify an automated incident response decision to a compliance auditor?

- Sample inefficiency
- Policy brittleness
- Explainability gap
- Sim-to-real gap

Answer: C **Explanation:** The explainability gap means that RL agents cannot easily articulate why a specific action was chosen, making it difficult to provide audit trails required by compliance frameworks such as SOC 2, ISO 27001, or financial regulations. Option A relates to training data needs. Option B relates to performance on novel scenarios. Option D relates to performance differences between simulation and production.

Q9. In Q-learning, what does the Bellman equation primarily accomplish?

- It calculates the exploration rate for the next episode
- It updates Q-values based on immediate reward plus discounted future value
- It determines the optimal neural network architecture for deep RL
- It selects the batch size for training

Answer: B **Explanation:** The Bellman equation is the core update rule: $Q(s,a)$ is updated based on the immediate reward plus the discounted estimate of future returns. This iterative updating is how Q-values converge to accurate estimates. Options A, C, and D describe unrelated configuration decisions.

Q10. A CISO is evaluating an RL-based honeynet system that learns which honeypot configurations attract sophisticated attackers versus automated scanners. What RL mechanism enables this adaptive deception?

- The agent fine-tunes its reward function based on attacker sophistication
- The agent's policy evolves to assign high Q-values to honeypot configurations that yield the most valuable intelligence signals
- The environment resets every episode, preventing the agent from retaining knowledge
- Multi-agent RL forces the honeypots to compete with each other

Answer: B **Explanation:** The agent observes which configurations lead to high-value outcomes (sophisticated attacker dwell time, credential telemetry) and reinforces those configurations in its policy via Q-value updates. Option A is incorrect — the reward function is fixed by design. Option C contradicts the premise of policy learning across episodes. Option D misapplies MARL.

Q11. Your organisation wants to deploy an RL incident response agent. The agent trains successfully in simulation with 95% accuracy but performs poorly in production with 60% accuracy. What is the MOST likely cause?

- The discount factor is too high
- The Q-table has too many states
- The sim-to-real gap: the simulation does not accurately replicate production network behaviour
- The agent was trained with too many epochs causing overfitting

Answer: C **Explanation:** The sim-to-real gap occurs when the simulated training environment differs from the real deployment environment in timing, traffic patterns, vendor behaviours, or edge cases. The agent learned a policy optimised for the simulation's characteristics that don't match production. Option A affects temporal reasoning but not accuracy gaps of this magnitude. Option B relates to scalability. Option D applies to neural networks, not RL policies in this context.

Q12. An RL agent for network defence uses an epsilon-greedy strategy with epsilon equal to 0.1 after convergence. What does this mean operationally?

- The agent explores 90% of the time and exploits 10% of the time
- The agent exploits its learned policy 90% of the time and explores randomly 10% of the time
- The agent applies a 10% discount to all future rewards
- The agent resets its Q-table every 10 episodes

Answer: B **Explanation:** In epsilon-greedy, epsilon is the probability of random exploration. With epsilon = 0.1, the agent chooses a random action 10% of the time (exploring) and follows its best-known policy 90% of the time (exploiting). Option A inverts the probabilities. Option C conflates epsilon with the discount

factor. Option D is not how epsilon-greedy works.

Q13. A security researcher uses multi-agent RL to train attackers and defenders simultaneously. After 50,000 episodes, the defender outperforms all known human-designed rule sets. What co-evolutionary phenomenon explains this?

- The defender agent memorised all attacker actions from the first episode
- The defender improved because each attacker improvement forced the defender to develop counter-strategies creating compounding capability growth
- The attacker agent deliberately reduced its effectiveness to allow the defender to succeed
- Multi-agent RL always converges to a human-optimal policy

Answer: B **Explanation:** Co-evolution drives mutual improvement: each time the attacker finds a new exploit, the defender must develop a counter. Each time the defender patches a weakness, the attacker must find new paths. This iterative pressure compels both agents to discover strategies far beyond initial human design. Option A misunderstands RL — agents don't memorise, they generalise. Option C contradicts the adversarial setup. Option D is false — MARL converges to Nash equilibrium, not necessarily human-optimal.

Q14. Which of the following BEST describes the purpose of the discount factor in RL?

- Controls how much the agent explores unknown actions
- Determines how strongly future rewards are weighted relative to immediate rewards
- Sets the learning rate for weight updates in neural networks
- Specifies the maximum number of episodes per training run

Answer: B **Explanation:** The discount factor (γ) is applied to future rewards in the Bellman equation. A γ of 0.99 means a reward one step in the future is valued at 99% of an immediate reward. $\gamma = 0$ makes the agent purely greedy. $\gamma = 1$ values all future rewards equally with immediate ones. Option A describes the exploration rate. Option C describes the learning rate. Option D is unrelated.

Q15. Why is RL considered particularly well-suited for cybersecurity problems compared to other ML paradigms?

- Security datasets are always labelled, which RL requires
- RL can operate without any data — it generates its own through experience
- Security involves sequential decision-making with intelligent adversaries and delayed consequences — exactly the problem structure RL is designed for
- RL models are faster to train than supervised models on security datasets

Answer: C **Explanation:** RL's design assumptions match security's problem structure: decisions are sequential, consequences are delayed, and adversaries are intelligent and adaptive. Supervised learning requires labelled data and does not model sequential adversarial dynamics. Option A is false — RL does not require labelled data. Option B is technically true but not the primary reason RL suits security. Option D is

false — RL typically requires far more compute to train.

Q16. A security AI team notices their RL-based SOAR agent keeps generating escalate-to-analyst actions even for confirmed low-severity events. What reward design flaw MOST likely caused this?

The agent was trained with too few episodes

The reward for escalation is positive or zero but never penalised, so the agent uses it as a safe default to avoid risk

The agent's Q-table is too small to represent all incident types

The discount factor is too low

Answer: B Explanation: If escalation carries no negative reward, the agent learns it is always safe — it never causes direct negative outcomes. The agent exploits this by escalating everything to avoid risk. The fix is to penalise unnecessary escalations by analyst time cost. Option A would cause underfitting, not systematic overescalation. Option C is a scalability issue. Option D affects temporal reasoning, not action preference patterns.

Q17. Scenario: A financial institution's RL-based network defence system failed to respond to a novel attack chain using an obscure protocol that was absent from simulation training data. A compliance auditor is asking why the system failed and what controls should be added. A financial institution deployed an RL-based network defence system. During a red team exercise, the team used a novel attack chain involving an obscure protocol not included in the simulation training environment. The RL defender failed to respond appropriately. Which response to the compliance auditor is MOST accurate?

The RL agent failed because its reward function was incorrectly designed

The RL agent failed due to policy brittleness caused by the sim-to-real gap; the novel attack was outside its training distribution. Compensating controls include human-in-the-loop escalation for low-confidence decisions and scheduled retraining with adversarial simulation expansion.

The RL agent failed because the neural network had too few layers

The RL agent failed because reinforcement learning is fundamentally unsuitable for financial sector security

Answer: B Explanation: This is a classic out-of-distribution failure. The agent's policy was optimised for attack patterns in its training simulation; the novel protocol was never seen, so Q-values for the correct response were never learned. Compensating controls must address both the gap (retraining with broader simulation) and the failure mode (human escalation for uncertain situations). Option A is plausible but incorrect — the reward function worked; the training distribution was the issue. Options C and D are not supported.

Q18. Scenario: A CISO at a healthcare organisation is evaluating an RL-based automated incident response system. The vendor claims the system learns and improves over time. The concern is that the system might autonomously isolate a medical device mid-procedure if the device is flagged as suspicious. A CISO at a healthcare organisation is evaluating an RL-based automated incident response system. The

concern is that the system might autonomously isolate a medical device mid-procedure. Which safeguard BEST addresses this while preserving RL's adaptive benefits?

Disable the RL system's ability to learn after initial deployment

Implement a human-in-the-loop constraint that prevents the RL agent from taking high-impact actions such as device isolation without explicit human approval, while allowing autonomous low-impact actions

Replace the RL system with a rule-based system for all healthcare environments

Set the discount factor to zero to prevent the agent from considering future consequences

Answer: B **Explanation:** Human-in-the-loop (HITL) constraints are the standard safeguard for high-stakes autonomous decisions. The RL agent retains adaptive learning capability for low-risk actions while critical decisions require human authorisation. This balances autonomy with safety. Option A eliminates the core benefit of RL. Option C overcorrects — rule-based systems cannot adapt to novel threats. Option D makes the agent myopic and less safe for sequential threats.

Q19. Scenario: A threat intelligence company is using multi-agent RL to generate novel attack scenarios for training blue teams. After 100,000 episodes, the red-team agent has discovered a persistent attack strategy that human red teamers had not previously documented. A threat intelligence company using MARL discovers a persistent attack strategy that human red teamers had not previously documented. What is the MOST appropriate next step?

Discard the discovered strategy as it may be unrealistic

Analyse the discovered strategy, validate it against real network environments, document it as a new threat model, and update defensive training data accordingly

Retrain the red-team agent to avoid discovering novel strategies

Patent the attack strategy to prevent other organisations from using it

Answer: B **Explanation:** MARL's primary research value is discovering strategies humans would not generate. The correct response is to rigorously validate the strategy, document it, and use it to improve defensive tooling and threat models. Option A wastes discovered intelligence. Option C actively harms the research objective. Option D is ethically problematic and impractical for attack techniques.

Q20. Scenario: An e-commerce company's RL-based fraud detection agent has been in production for six months. Analysts noticed it is blocking a new category of legitimate transactions from a payment processor that launched three months ago. An e-commerce company's RL-based fraud detection agent has been in production for six months. Analysts notice the agent is blocking legitimate transactions from a payment processor that launched three months ago. What is the MOST likely explanation and correct remediation?

The agent is exploring randomly and will self-correct within hours

The agent's training data predates the new payment processor; its transaction patterns fall outside the agent's learned state space and are flagged as anomalous. Remediation: retrain or fine-tune with labelled examples of legitimate transactions from the new processor.

The Q-table has run out of space for new states

The discount factor is too high causing over-indexing on preventing fraud

Answer: B **Explanation:** RL agents fail on out-of-distribution inputs. A payment processor that launched after training is completely outside the agent's learned state distribution. Its transaction patterns are novel and therefore assigned low Q-values for allow, causing blocks. Remediation requires exposing the agent to labelled examples of the new processor's legitimate traffic. Option A is incorrect — converged production agents do not spontaneously self-correct. Options C and D are technical misunderstandings.